

Basics of Robotics Instructor Handbook

Everything you need to run the session, plus answers to the questions students actually ask. Session 01, first-time students, no prior electronics.

Instructor: MD. Monjurul Hasan Bhuiyan · BRACU Mongol Tori, Tivoniq
Deck: [~/robotics-lecture/index.html](#) · 35 slides · runs offline in Chrome
Survey: claude.ai artifact, share with edit access before class

Contents

- | | |
|--|--------------------------------------|
| 1. Thirty minutes before class | 9. Questions: sensors, motors, power |
| 2. Driving the deck | 10. Questions: communication |
| 3. The warm-up survey | 11. Questions: the practical stuff |
| 4. Run sheet, slide by slide | 12. Numbers worth memorising |
| 5. The seven coding exercises | 13. When something breaks |
| 6. Questions students ask: electricity | 14. Glossary for beginners |
| 7. Questions: the board and the chip | 15. Homework and next session |
| 8. Questions: code | |

The one-line summary of the whole session

A robot senses, decides and acts, over and over. A microcontroller is the part that decides. Today they write the deciding part themselves, watch it move something, and learn why a pin cannot drive a motor on its own.

The three things they must leave with

1. **setup runs once, loop runs forever.** Everything else is detail.
2. **A pin is either 5 V or 0 V.** Analog output is a trick played with speed.
3. **Grounds must be shared and pins cannot drive motors.** These two facts prevent most burnt hardware.

1. Thirty minutes before class

On the laptop

1. Open `~/robotics-lecture/index.html` in Chrome. Press `F` for full screen.
2. Click through slides 18 to 24 once and press **Upload** on each. This warms the fonts and confirms every exercise runs.
3. Click anywhere on the page once. Chrome blocks audio until a page gets a click, and the buzzer on the LEDs exercise needs it.
4. Set the display to mirror, not extend, unless you are comfortable with two screens.
5. Turn off notifications and sleep. A notification banner over the projector is the fastest way to lose a room.

The survey

1. Open the survey artifact and share it from the page's share menu **with edit access**. Answers only save for viewers who can write.
2. Open it a second time in a separate window and press **Show the results screen**. Leave that window on a second desktop so you can switch to it during the warm-up.
3. Test it from your own phone. If it does not save, fall back to the tap-to-count poll on slide 3 and move on. Do not spend class time debugging it.

In the room

- Write the deck link on the board so latecomers can follow on their laptops.
- Check the projector brightness on slide 18. The code needs to be readable from the back row. If it is not, sit in the back row yourself and check before anyone arrives.
- If you brought real hardware, keep it switched off and out of reach until you want their attention on it.

The failure mode of a first workshop

Not that the material is too hard. It is that twenty minutes disappear into installing software on twenty different laptops. That is why this deck runs the Arduino in the browser. Resist any urge to have them install the real IDE today. Tell them it is next session's first task.

2. Driving the deck

Key	What it does
→ or space	Next slide
←	Previous slide
0	Grid of all 35 slides, click one to jump. Use this when a question jumps ahead.
F	Full screen
S	Short version. Skips the two optional slides (16 Power, 28 Other signals).
B	Blank the screen. Use it when you want every eye on the hardware or on you.
Home / End	First and last slide
Click left or right quarter	Previous and next, if you are away from the keyboard

The clock in the bottom left starts when you click it. Click it as you say your first sentence. The chapter name sits next to it so you always know where you are without looking at the slide.

Slides that need you to do something

Slide	Action
3 Warm-up	Tap the hand-count buttons as hands go up. Switch to the results window after the QR has been up for two minutes.
5 Robot anatomy	It cycles by itself. Click a circle to hold on one part.
6 Sensing	Pick each sensor, drag the slider, let them call out what the number does.
10 Inside the chip	Hover the blocks. Spend longest on SRAM, it is the one that bites them later.
12 MCU family	Click through the tabs. This is where you mention ESP32 and why we are not on it today.
14, 15 Board and pins	Hover the parts. Ask them to find the reset button before you highlight it.
18 to 24	Type live. Break the code on purpose at least once.
27 PWM	Drag the slider slowly from 0 to 255 and let them watch the fan and LED together.
30 H-bridge	Press Forward, Reverse, Brake. Ask what happens if both top switches close.
33 The buses	Press Send a byte on each tab. The scope at the bottom shows the wires.
34 Which bus	Read each situation out, let them answer, then tap to reveal.

3. The warm-up survey

Five questions. Two of them have a right answer and most rooms get both wrong, which is the point: it buys you their attention for the next hour.

Question 3: which is hardest for a robot today?

Answer: folding a towel. Climbing stairs is solved, chess has been solved since 1997, face recognition is a phone feature. Folding a towel is not, because a towel has effectively infinite shapes and no fixed points to grip. This is Moravec's paradox: the things humans find hard (chess, arithmetic) are easy for machines, and the things any two-year-old can do (grasping, balance, recognising a crumpled object) are extremely hard.

Say: The hardest problems in robotics are not the ones that look impressive. They are the ones your little sister solves without thinking.

Question 4: a decent robot arm costs less than a flagship phone

Answer: true. A hobby 6-axis arm on hobby servos runs in the range of a mid-range phone. A phone flagship costs more than a lot of usable robotics hardware. What is expensive in robotics is precision, repeatability and the engineer's time, not the parts.

If the survey fails

Use the tap-to-count poll on the same slide, ask the two questions out loud, and take a show of hands. It takes ninety seconds and it works with no network at all.

4. Run sheet, slide by slide

Times assume a two-hour session with questions. The Short version key removes about eight minutes.

START · 12 MINUTES

- | | | |
|-----------|---|-------|
| 01 | Welcome
<div>Say: By the end of today every one of you will have written code that made something move. Not watched me do it. Written it.</div> | 1 min |
| <hr/> | | |
| 02 | Your instructor
Keep it short and concrete. Mongol Tori, what a rover has to do, and one sentence on Tivoniq as sensors in a field sending data to a server. The point is credibility, not biography. | 2 min |
| <hr/> | | |
| 03 | Warm-up
Speed round of introductions, one sentence each, keep them moving. QR up. Switch to the results window and discuss the two trick questions. | 7 min |
| <hr/> | | |
| 04 | Roadmap
Tell them the shape of the session so nobody is wondering when the fun part starts. | 2 min |

WHAT A ROBOT IS · 14 MINUTES

- | | | |
|-----------|---|-------|
| 05 | Five parts
<div>Ask: Is a washing machine a robot? Let them argue for a minute. Answer: it senses, decides and acts, so by this definition yes. What it lacks is the ability to change what it does. That is the interesting line.</div> | 4 min |
| <hr/> | | |
| 06 | Sensing
Drag each slider. Land the idea: every sensor ends as a number, and your code never sees light or heat, only numbers. | 4 min |
| <hr/> | | |
| 07 | Acting
Click all four. Point out that the code for all of them is the same shape: write a number to a pin. | 3 min |
| <hr/> | | |
| 08 | The loop
The runner dot goes around while the line-follower steps light up. This is the mental model for the whole day. | 3 min |

THE BRAIN · 18 MINUTES

- | | | |
|-----------|---|-------|
| 09 | Three kinds of computer
<div>Say: Your laptop is a thousand times faster and completely useless for this, because it cannot promise to do something at an exact microsecond.</div> | 4 min |
| <hr/> | | |
| 10 | Inside the chip
Hover each block. Spend time on flash versus SRAM, because "why did my program get weird when I added a String" comes from here. | 5 min |
| <hr/> | | |
| 11 | 16 MHz and 32 KB
Slow the tick down to 1 Hz and let them watch it, then say the real chip does that sixteen million times a second. | 3 min |

12	The board family	4 min
----	-------------------------	-------

Be honest here: the UNO is the teaching board, the ESP32 is what you would actually build with. Say why we start here anyway: it forgives 5 V mistakes and every concept transfers.

13	Why Arduino	2 min
----	--------------------	-------

Show the two code blocks side by side. They do the same thing. One of them is readable.

THE BOARD · 10 MINUTES

14	Every part of the board	5 min
----	--------------------------------	-------

Before you click anything, ask them to find the chip that runs their code. Most will point at the 16U2 or the crystal.

15	Pin map	4 min
----	----------------	-------

The three facts that matter: 0 and 1 are the serial pins, ~ means PWM, A4 and A5 are also I2C.

16	Power (optional)	3 min
----	-------------------------	-------

Press "Show me" on the brown-out demo. Skip this slide if you are behind; it comes back on slide 29 anyway.

WRITING CODE · 45 MINUTES, THE HEART OF THE SESSION

17	The workbench	3 min
----	----------------------	-------

Explain the four panels, then get out of the slide quickly. They learn by typing, not by hearing about typing.

18-24	Seven exercises	~6 min each
-------	------------------------	-------------

Full detail in the next section. Rhythm for each one: press Upload, let them see it work, then break it deliberately, read the error out loud, fix it, then give them the change to make.

SIGNALS · 16 MINUTES

25	Digital and analog	4 min
----	---------------------------	-------

Drag the source slider across 3 V and show that digitalRead flips while analogRead moves smoothly.

26	Logic levels	4 min
----	---------------------	-------

Park the slider in the undefined band and let it sit there while you explain why that is worse than being wrong.

27	PWM	5 min
----	------------	-------

The single most useful slide in the deck. Move the slider slowly. Say the pin is never at 2.5 V at least twice.

28	Other signals (optional)	3 min
----	---------------------------------	-------

Frequency, pulse width, quadrature. Skip if short on time; mention encoders exist.

MOTORS · 12 MINUTES

29	Why a driver	5 min
----	---------------------	-------

The counting bars land the point. Press "Cut the power" for the spike.

30	H-bridge	4 min
Ask what happens if Q1 and Q3 close together before you say the words shoot-through.		
31	Four motors	3 min
Tie each one to something they have seen: printer, arm, 3D printer, drone.		
TALKING · 12 MINUTES		
32	Why buses	3 min
Flip between the two wiring pictures. Fourteen wires versus two.		
33	The buses	6 min
One send per tab. Do not try to teach the protocols properly, just the shape of each.		
34	Which bus	3 min
Quiz format. Read, pause, reveal.		
CLOSE · 4 MINUTES		
35	Wrap-up	4 min
Read the eight things back to them. Give the homework. Say the line about not getting stuck alone at 2 am and mean it.		

5. The seven coding exercises

Each slide has an editor, a serial monitor, a drawn UNO wired to a breadboard, and a wiring list. **Verify** compiles without running. **Upload** runs it. The simulator supports setup and loop, variables, arrays, if, else, for, while, your own functions, pinMode, digitalWrite, digitalRead, analogRead, analogWrite, delay, millis, map, constrain, tone, noTone, the Serial family, and Servo.

Teaching rhythm that works

1. Upload it as-is. Let them see it work before any explanation.
2. Break it on purpose: delete a semicolon, press Upload, read the error out loud.
3. Fix it, explain the one new idea.
4. Give them the change to make and wait. Silence while they think is not wasted time.

18 • Make a light blink

```
void setup() {  
  pinMode(9, OUTPUT);  
}  
void loop() {  
  digitalWrite(9, HIGH);  
  delay(500);  
  digitalWrite(9, LOW);  
  delay(500);  
}
```

New idea: a pin is an output you switch between 5 V and 0 V, and delay stops everything.

Their change: both delays to 100. It blinks five times faster.

Push further: set both to 5 and ask what they see. The LED looks dim and steady, not blinking. That is the whole idea of PWM arriving seven slides early, so name it and move on.

Wiring on screen: pin 9 to a 220 Ω resistor, resistor to the LED's long leg, short leg to the ground rail, ground rail back to GND on the board.

19 • Four lights, one loop

New idea: an array holds the four pin numbers and a for loop walks through them, so four LEDs cost the same code as one.

Their change: run it backwards (`for (int i = 3; i >= 0; i--)`) or add `tone(12, 1200)` for the buzzer. The buzzer makes real sound in the browser.

Watch for: students writing `i <= 4`. It reads pin `leds[4]`, which does not exist. In this simulator it is harmless; on a real chip it reads whatever byte is next in memory. Good moment to explain that C++ does not check array bounds for you.

20 • Make a decision

New idea: digitalRead, and if / else. Sense, decide, act, in five lines.

Their change: invert it so the light is on until you press.

The pull-down question will come up. Answer: with nothing connected, an input pin is like a loose wire, picking up noise from the air. The 10 kΩ resistor to ground holds it at 0 V until the button connects it to 5 V. If you use `INPUT_PULLUP` instead, the chip's own resistor holds it at 5 V and a press pulls it to 0, so the logic inverts.

21 • Make the board talk

New idea: Serial is the only window into what the chip is thinking, and it goes both ways.

Their change: type `1` and `0` into the box under the console.

Add if they are quick: a third command that blinks three times.

Point out: `Serial.read()` gives one character, and `'1'` with quotes is a character while `1` without quotes is a number. They are not the same thing.

22 • Read the knob

New idea: `analogRead` turns a voltage into 0 to 1023.

Their change: drag the knob and read the numbers.

Ask: what voltage is 512? Answer: about 2.5 V. What is one step worth? 5 V over 1024 steps, about 4.9 mV.

23 • Dim a light with a knob

New idea: `map` rescales one range to another, and `analogWrite` fakes an analog output.

Their change: flip the `map` so it gets brighter the other way: `map(knob, 0, 1023, 255, 0)`.

Watch for: using a pin without a `~`. The simulator prints a warning in the console rather than failing silently, which is kinder than the real IDE.

24 • Move something in the real world

New idea: a library hides work someone else already did. `Servo.h` handles microsecond pulse timing so they can just say an angle.

Their change: replace the loop body with `myServo.write(90);` so it holds still, then drive it from the knob:

```
void loop() {
  int knob = analogRead(A0);
  int angle = map(knob, 0, 1023, 0, 180);
  myServo.write(angle);
}
```

Say while it sweeps: on a real bench, power that servo from its own supply and join the grounds. This is the sentence that saves their first project.

6. Questions students ask: electricity

Why does the LED need a resistor? It worked without one when my friend tried.

An LED is a diode, not a resistor. Above its forward voltage, around 2 V for a red one, the current rises almost vertically with voltage. Straight onto 5 V it will pull far more than it can survive. The 220 Ω resistor sets the current: $(5 - 2) \div 220 \approx 13.6$ mA. It may well light without one for a while. It is dying while it does.

Which leg of the LED is positive?

The longer leg is the anode, the positive side. The shorter leg is the cathode and the plastic body has a flat spot on that side. Current flows in the long leg and out the short one.

What if I put the LED in backwards?

Nothing. It just does not light. At 5 V you will not damage a normal LED by reversing it. This is the single most common reason a beginner's circuit does nothing.

What actually is voltage? What is current?

Voltage is the push, current is how much is flowing. The classic picture is water in a pipe: voltage is the pressure, current is the flow rate, and resistance is how narrow the pipe is. Ohm's law ties them: $V = I \times R$. The useful habit is that voltage is always measured *between* two points, which is why ground matters.

Why do the grounds have to be connected?

Because voltage is a difference, not an absolute. If your Arduino and your motor supply do not share a ground, "5 V" on one has no defined relationship to "5 V" on the other, and the signal between them means nothing. Two supplies with separate grounds is probably the most common wiring bug in beginner robotics.

What is a floating pin?

An input pin with nothing connected. It is not 0 and not 5, it is whatever charge happens to be sitting on it, and it will pick up interference from your hand, the mains, anything. It reads randomly. A pull-up or pull-down resistor fixes it by gently holding it at a known level.

What happens if I connect 5 V straight to GND?

A short circuit. On USB power, the board or the computer's port protection trips and the board goes dark until you unplug it. On a battery, nothing protects you: wires get hot, the battery can vent, and LiPo batteries can catch fire. Always check before applying power.

Why 5 V for one board and 3.3 V for another?

Smaller manufacturing processes cannot survive 5 V, and lower voltage uses less power. So newer chips run at 3.3 V. Putting 5 V into a 3.3 V input can destroy it, sometimes immediately and sometimes weeks later. Going the other way, 3.3 V into a 5 V input, usually reads as HIGH because the threshold is about 3 V, but "usually" is not a design.

What is a capacitor for?

It stores a small amount of charge and gives it back quickly. Near a motor or a servo it acts as a local reservoir, so when the motor suddenly demands current the voltage does not dip enough to reset the chip. That is why boards are covered in them.

7. Questions: the board and the chip

Is Arduino the chip or the board?

Neither, exactly. The chip is an ATmega328P made by Microchip. Arduino is the board it is mounted on, plus the IDE, plus the library that turns register writes into `digitalWrite()`, plus the bootloader in the chip that lets plain USB upload code. Arduino is the ecosystem that made that chip usable by a first-year student.

What is a bootloader?

A small program already in the chip that runs for a moment at every reset. It listens on the serial line, and if a computer is trying to send a new sketch it writes it into flash. Without it you would need a hardware programmer and the ICSP header.

Where does my code actually live?

In flash, 32 KB of it, which keeps its contents with the power off. That is why the board runs your sketch the instant it gets power, with no boot process. Your variables live in SRAM, 2 KB, which is wiped on every reset.

Only 2 KB of RAM? My phone has 8 GB.

Four million times more, yes. It does not matter, because the chip is not storing photos. It holds a few dozen variables and does the same job forever. You run out of RAM only when you start doing things it was not built for, like holding long text strings.

Why is the L LED already blinking when I plug in a new board?

Boards ship with the Blink example loaded. The L LED is wired to pin 13, so Blink works with nothing attached.

What is the crystal for?

It sets the chip's clock, 16 million ticks a second, and it is accurate. Without an accurate clock, `delay(1000)` would not be a second and serial communication would drift out of sync and produce garbage.

What does the reset button do?

Restarts your sketch from the first line of setup. It does not erase anything. The board does exactly this by itself every time you upload.

Should I buy an UNO or an ESP32?

If you want to learn and you want mistakes to be survivable, UNO. If you want to build something real, ESP32: faster, far more memory, wifi and Bluetooth built in, usually cheaper. It is 3.3 V, so it is less forgiving. Learn on one, build on the other. Everything you learn today transfers.

Is a clone board fine?

Yes, functionally. The difference is the USB chip: official boards use an ATmega16U2, clones usually a CH340. The CH340 sometimes needs a driver install on Windows or macOS. That is the only practical difference you will notice.

Can I break the board by writing bad code?

Mostly no. Software cannot melt anything on its own. The exceptions involve hardware: setting two pins as outputs and wiring them together, driving more current than a pin can give, or connecting something to the wrong pin. The chip itself you can reprogram thousands of times.

8. Questions: code

Why is there no main()?

There is one, Arduino just writes it for you. It calls `setup()` once, then `loop()` over and over inside a `while(true)`. If you look in the Arduino core source you can find it.

What happens if I put my code in setup instead of loop?

It runs once and stops. The board is not off and has not crashed, it has simply finished. Reset it and it runs once more.

Why does everything need a semicolon?

Because C++ ignores line breaks. The semicolon is how you say "this statement is finished". A missing one is the most common compile error there is, and the compiler usually points at the *next* line, which is why beginners look in the wrong place.

What is the difference between = and ==?

One equals sign assigns: `x = 5` puts 5 into x. Two compares: `x == 5` asks whether x is 5. `if (x = 5)` compiles fine, silently sets x to 5, and is always true. This bug can cost you an evening.

Why does delay() freeze everything?

Because it does exactly what it says: it sits and counts. Nothing else in your sketch runs while it waits, so a button pressed during a delay is missed. The grown-up alternative is to check `millis()`:

```
unsigned long last = 0;
void loop() {
  if (millis() - last >= 500) {
    last = millis();
    // do the thing that used to be after delay(500)
  }
  // other code keeps running here
}
```

Do not teach this today unless someone asks. Mention that it exists and that it is next session.

What is the difference between int, float and long?

On an UNO, `int` is a whole number from -32768 to 32767. `long` is a much bigger whole number, which is why `millis()` returns one. `float` has a decimal point but is slow on this chip because there is no floating-point hardware. Prefer whole numbers when you can.

Why is my LED brightness backwards?

Either the map arguments are swapped, or the LED is wired to 5 V instead of to ground, in which case writing HIGH turns it off. Both are common and both are easy to spot once you know to look.

What does #include actually do?

It pastes another file into yours before compiling. `#include <Servo.h>` brings in the declarations for the Servo class so the compiler knows what `myServo.write()` means.

What is the difference between a function and a library?

A function is one named block of code. A library is a bundle of them, written and debugged by someone else, that you install once and include when you need it.

9. Questions: sensors, motors, power

How much current can a pin give?

20 mA is the recommended figure, 40 mA is the absolute maximum before damage, and the whole chip should stay under about 200 mA in total. An LED at around 15 mA is fine. Anything with a coil in it is not.

Why can't I just connect the motor to the pin?

A small DC motor wants 300 to 1000 mA running and can pull 2 A or more the moment it stalls. That is fifty times what the pin can give. The pin burns out, often taking the chip with it. A driver is a switch: the small signal from the pin controls it, and the big current from the battery goes through it, never through the pin.

What is back-EMF?

A motor is a coil, and a coil resists any change in the current through it. When you switch the current off, the coil keeps pushing and generates a large reverse voltage spike, easily tens of volts from a 6 V motor. That spike is what actually kills pins. Motor drivers have diodes inside to absorb it, which is also why a bare relay needs a flyback diode.

Can I power a servo from the 5 V pin?

A small unloaded one, briefly, sometimes. Under load it will pull several hundred milliamps, the 5 V rail sags, the chip browns out and resets, and your project looks like it has a software bug. Give motors and servos their own supply and join the grounds.

What is the difference between a servo and a normal motor?

A servo is a DC motor, a gearbox, a position sensor and a small controller in one case. You command an angle and it drives itself there and holds against a load. A plain DC motor just spins while you feed it, with no idea where it is.

Why does my servo jitter?

Usually power: the supply sags every time it moves. Sometimes it is signal noise on a long wire, and cheap servos hunt around the target angle by nature. Separate supply, shared ground, short signal wire.

How do I make a motor go backwards?

Reverse the current through it, which is what an H-bridge does. Four switches: close two opposite corners for one direction, the other two for the other. Close both on the same side and you short the battery through the driver, which is called shoot-through and is why you use a driver chip instead of four loose transistors.

How do I control motor speed?

PWM on the driver's enable pin. The motor's own inertia averages the switching out, so 50% duty gives you roughly half speed. Note that a motor will not usually start at very low duty cycles; it needs enough to overcome friction.

Can I read a sensor that is far away?

Analog voltage over a long wire picks up noise and drops along the resistance of the cable, so it degrades. This is exactly why buses like RS-485 and CAN exist: they send a difference between two wires, so noise that hits both cancels out.

10. Questions: communication

What is baud rate really?

Bits per second on the wire. UART has no clock line, so the receiver cannot see how fast the sender is going; it simply assumes the rate you configured and samples at those moments. If the two disagree, the bits are sliced at the wrong times and you get garbage characters. 9600 is convention, not magic. 115200 is common now.

Why do TX and RX have to cross?

Because one board's transmit is the other board's receive. TX to RX, RX to TX, and a shared ground. Wiring TX to TX is the other classic beginner failure.

What is an I2C address, and why do two devices clash?

Every I2C device answers to a number, usually 7 bits, set by the manufacturer. Two devices with the same address on the same bus both answer at once and the data is nonsense. Many modules have a solder jumper or an address pin to shift to an alternative.

Why does I2C need pull-up resistors?

Devices on I2C can only pull the line down, never drive it up. Something has to return the line to 5 V, and that is the pull-up, typically 4.7 kΩ. Most breakout boards already have them fitted, which is why it often works with none added.

SPI or I2C?

SPI when you need speed: it is much faster and full duplex, but every device needs its own chip-select pin. I2C when you need few wires: two lines shared by many devices, at the cost of speed. Screens and SD cards tend to be SPI, small sensors tend to be I2C.

Why do cars use CAN instead of something simpler?

Because it degrades well. Any node can talk, and if two start at once the more urgent message wins automatically while the other retries, with no master to fail. It is differential, so it survives electrical noise. A car keeps running when one module dies.

Is USB just serial?

Not really, but on an UNO the effect is close. The 16U2 chip on the board converts USB into plain serial for the main chip. That is why your computer sees a COM port and why the Serial monitor works over the same cable that powers the board.

11. Questions: the practical stuff

Is this a real Arduino on the screen?

It is a simulator: a small interpreter that runs a useful subset of Arduino C++ and a model of the hardware. The code you write here compiles and uploads to a real UNO with no changes. What it does not simulate is anything analog and messy: exact timing, noise, brown-outs, or a component wired backwards.

Do I need to buy anything to practise?

Not to practise the code. A starter kit with an UNO, a breadboard, jumper wires, LEDs, resistors, a couple of buttons and one servo covers everything in this session. Share one between two or three people if budget is tight; pairs learn faster than individuals anyway.

Where do I buy parts?

Fill in your own local recommendation here before class: one shop and one online seller you trust, with a rough price for a starter kit. Students will ask and a vague answer wastes the moment.

Do I need to be good at maths?

For this, no. You need Ohm's law and the ability to divide. Maths becomes real when you get to control, kinematics and filtering, and by then you will want it, because you will have a robot that wobbles and you will want to know why.

Is robotics only hardware people?

No. Every robot needs mechanical design, electronics, firmware, higher-level software, and someone who can write the documentation and win the funding. Rover teams lose more often on integration and paperwork than on engineering.

What should I learn next?

In order: millis instead of delay, then a sensor with a library over I2C, then a motor driver with real motors, then a small project that does one thing end to end. After that, ESP32 and PlatformIO, and then ROS 2 if you are going toward autonomy.

How do I join Mongol Tori?

Your answer, not mine. Have the recruitment route and the next deadline ready, because after a session like this two or three of them will ask.

12. Numbers worth memorising

The Arduino UNO

Thing	Number
Chip	ATmega328P, 8-bit
Clock	16 MHz, one tick every 62.5 ns
Flash (your program)	32 KB, minus about 0.5 KB for the bootloader
SRAM (your variables)	2 KB
EEPROM (survives power off)	1 KB, about 100,000 writes per cell
Digital pins	14, numbered 0 to 13
PWM pins	6: 3, 5, 6, 9, 10, 11, marked with ~
Analog inputs	6: A0 to A5, 10-bit, 0 to 1023
ADC resolution	$5\text{ V} \div 1024 \approx 4.9\text{ mV}$ per step
Current per pin	20 mA recommended, 40 mA absolute maximum
Current for the whole chip	about 200 mA total
5 V pin can supply	roughly 500 mA on USB power
Input voltage via jack	7 to 12 V
PWM frequency	about 490 Hz on pins 3, 9, 10, 11; about 980 Hz on 5 and 6
I2C pins	A4 is SDA, A5 is SCL
SPI pins	11 MOSI, 12 MISO, 13 SCK, 10 usually CS
Serial pins	0 RX, 1 TX, shared with USB

Logic levels

Range on a 5 V board	The chip reads
0 V to about 1.5 V	LOW, logic 0
about 1.5 V to about 3.0 V	Undefined. Could read either. Avoid.
about 3.0 V to 5 V	HIGH, logic 1

Typical current draw

Part	Current
One LED with a 220 Ω resistor	about 15 mA
The UNO itself, idle	about 45 mA
Small hobby servo, moving	300 to 700 mA

Small DC motor, running	300 to 1000 mA
The same motor, stalled	2000 mA and up

The buses at a glance

Bus	Wires	Speed	Devices	Use it for
UART	2 (TX, RX)	up to ~1 Mbit/s	exactly two	board to board, GPS, Bluetooth
I2C	2 shared	100 / 400 kbit/s	up to 127	small sensors, OLED screens, RTC
SPI	4 (+1 CS each)	8 to 50 Mbit/s	many	SD cards, TFT screens, radios
1-Wire	1	~16 kbit/s	many	DS18B20 temperature sensors
RS-485	2 differential	up to 10 Mbit/s	up to 32	long cable runs, factories, Modbus
CAN	2 differential	up to 1 Mbit/s	many, no master	vehicles, rover drive systems

Resistor colour code, first three bands

Black 0, brown 1, red 2, orange 3, yellow 4, green 5, blue 6, violet 7, grey 8, white 9. The third band is the number of zeros. So red-red-brown is 2, 2, one zero, which is 220 Ω .

13. When something breaks

In the deck, during class

Symptom	Do this
A slide looks empty or half drawn	Press ← then → to re-enter it. Everything rebuilds on entry.
The buzzer makes no sound	Click once anywhere on the page. Chrome blocks audio until a page is clicked.
Typing feels wrong in the editor	Click into the code area once. The cursor may be in the serial input box.
The code will not run	Read the red line in the console. It gives the line number. Press Reset to restore the original sketch.
Fonts look wrong	No internet. The deck falls back to system fonts and still works. Ignore it.
Everything is too small at the back	Press F for full screen. The stage scales to the window.
You are running long	Press S. It skips the two optional slides. Then cut exercise 19 and slide 28.

On real hardware, when they get boards

Symptom	Cause and fix
Board does not appear as a port	Charge-only USB cable, or a missing CH340 driver on a clone. Try another cable first, it is the cable more often than not.
<code>avrdude: stk500_recv(): programmer is not responding</code>	Wrong port or wrong board selected, or something is wired to pins 0 and 1. Unplug those two pins and try again.
Upload works but nothing happens	Code is in setup instead of loop, or the LED is in backwards, or the pin number in code does not match the wiring.
Board resets randomly	Power. Something is drawing more than the supply can give, usually a servo or motor on the 5 V pin.
Serial monitor shows garbage	Baud mismatch. Set the monitor to the same number as <code>Serial.begin()</code> .
Sketch is too big / low memory warning	Long strings in flash, or too many libraries. Move constant text into <code>F("...")</code> .
Board gets hot	Unplug immediately. Short circuit, or a supply on the wrong pin.

If a student's board dies in a future class

Do not let them feel stupid about it. Everyone in robotics has killed hardware. Find out what it was wired to, explain the mechanism, and move on. The lesson is the mechanism, not the loss.

14. Glossary for beginners

Word	What it means, in one line
Actuator	Anything that turns a signal into movement, light or sound.
ADC	Analog to digital converter. Turns a voltage into a number.
Anode / cathode	Positive and negative legs of a diode or LED.
Back-EMF	The voltage spike a coil throws back when you cut its current.
Baud	Bits per second on a serial line.
Bootloader	Small program in the chip that lets USB upload your sketch.
Breadboard	A board full of connected holes for building a circuit without soldering.
Brown-out	Supply voltage dipping far enough that the chip resets.
Duty cycle	The fraction of each PWM period the pin is high.
EEPROM	Small memory that keeps its contents with the power off.
Firmware	The program running on a microcontroller.
Flash	Where your compiled program is stored on the chip.
Floating pin	An input connected to nothing, reading noise.
GPIO	General purpose input output. An ordinary pin.
Ground	The 0 V reference every voltage is measured against.
H-bridge	Four switches that let you run a motor both ways.
Library	Someone else's debugged code that you include and use.
MCU	Microcontroller. A whole small computer on one chip.
Pull-up / pull-down	A resistor holding an input at a known level until something changes it.
PWM	Switching a pin on and off fast so it behaves like a voltage in between.
Sketch	Arduino's word for a program.
SRAM	The working memory your variables live in.
Stall current	What a motor draws when it is jammed. The worst case for your supply.

15. Homework and next session

What you gave them

1. Blink an LED in a pattern they invent, not one they copy.
2. Read the knob and print the value to the Serial monitor.
3. Make the button start a servo sweep.

All three can be done in the browser on this deck, so nobody is blocked by not owning hardware. Tell them that explicitly.

What to line up for session two

- Install the real Arduino IDE, first thing, with you in the room to unblock drivers.
- `millis()` instead of `delay()`, and why blocking code cannot be a robot.
- One real sensor over I2C, so they see a library and an address for the first time.
- A motor driver with real motors, with the power rules from today enforced.
- Finish with a line follower or an obstacle avoider, so they leave with a robot that does one complete thing.

The thing to say at the end

They will forget the pin numbers. What you want to survive the week is that a robot is a loop that senses, decides and acts, that they can write the deciding part, and that when it does not work it is a fault to find, not a verdict on them.

Deck reference

Run: `~/robotics-lecture/index.html` in Chrome, press F.

Keys: arrows move, O all slides, F full screen, S short version, B blank screen.

Survey: share the artifact with edit access, open a second window for the results screen.

Files: `README.md` in the same folder has the technical detail.